LAB ASSIGNMENT – 10.1

Name: Ch Vishwanath Rao

Batch:02

Ht no: 2303A51095

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty

Python script.

Sample Input Code:

Calculate average score of a student

def calc_average(marks):

total = 0

for m in marks:

total += m

average = total / len(marks)

return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks)

Expected Output:

• Corrected and runnable Python code with explanations of the fixes.

CODE:

```
def calc_average(marks):
    if not marks:
        raise ValueError("Marks list cannot be empty")

    total = sum(marks)
    return total / len(marks)

marks = [80, 70, 90, 60, 85]
print(calc_average(marks))
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai> & C:/Python314/python.exe "c:/Users/chvis/OneDrive/Desktop/ai/all git pdf
77.0
PS C:\Users\chvis\OneDrive\Desktop\ai>
```

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style

guidelines.

Sample Input Code:

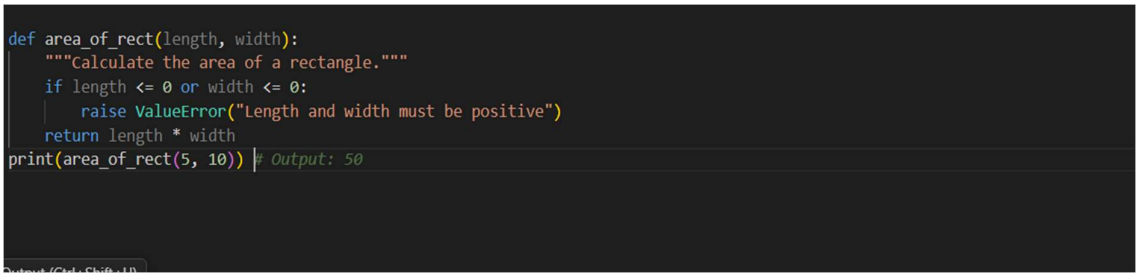
def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))

Expected Output:

• Well-formatted PEP 8-compliant Python code.

CODE:



```python
def area_of_rect(length, width):
    """Calculate the area of a rectangle."""
    if length <= 0 or width <= 0:
        raise ValueError("Length and width must be positive")
    return length * width
print(area_of_rect(5, 10)) # Output: 50
```

Output:

```
PS C:\Users\chvis\OneDrive\Desktop\ai> & C:/Python314/python.exe "c:/Users/chvis/OneDrive/D
5000
PS C:\Users\chvis\OneDrive\Desktop\ai>
```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its

logic.

Sample Input Code:

def c(x,y):

return x*y/100

a=200

b=15

print(c(a,b))

Expected Output:

• Python code with descriptive variable names, inline

comments, and clear formatting.

CODEO

```
def calculate_percentage(value, total):
    """Calculate percentage of a value relative to a total."""
    if total == 0:
        raise ValueError("Total cannot be zero.")
    return (value / total) * 100


a = 500
b = 150
print(calculate_percentage(a, b))
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai> & C:/Python314/python.exe "c:/Users/chvis/OneDrive/Desktop/ai/ai
Y/demo.py"
333.33333333333337
PS C:\Users\chvis\OneDrive\Desktop\ai>
```

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable

functions.

Sample Input Code:

students = ["Alice", "Bob", "Charlie"]

print("Welcome", students[0])

print("Welcome", students[1])

print("Welcome", students[2])

Expected Output:

• Modular code with reusable functions.

CODE:

```
#refactor the below code with modular code with reusable functions and error handling
def add_student(students, student_name):
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

```
def add_person(people_list, person_name):
    """Add a person to the list if not already present."""
    if not person_name or not isinstance(person_name, str):
        raise ValueError("Person name must be a non-empty string")
    if person_name.upper() in [x.upper() for x in people_list]:
        raise ValueError(f"Person {person_name} already exists")
    people_list.append(person_name)
    return people_list

def display_people(people_list):
    """Display all people in the list."""
    if not people_list:
        print("No people in the list")
        return
    print("People:", people_list)

try:
    people_list = ["ALICE", "BOB", "CHARLIE"]
    display_people(people_list)
    print("Welcome", people_list[0])
    print("Welcome", people_list[1])
    print("Welcome", people_list[2])
except ValueError as e:
    print(f"Error: {e}")
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai> & C:/Python314/python.exe "c:/Users/chvis/OneDrive/Desktop/ai/all
Y/demo.py"
People: ['ALICE', 'BOB', 'CHARLIE']
Welcome ALICE
Welcome BOB
Welcome CHARLIE
```

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

Find squares of numbers

nums = [i for i in range(1,1000000)]

squares = []

for n in nums:

squares.append(n**2)

print(len(squares))

Expected Output:

• Optimized code using list comprehensions or vectorized

operations.

CODE:

```python
#refactor the below code with the list comprehension, error handling and input validation
data = [i for i in range(1, 1000000)]
results = []
results = [x**2 for x in data]
for x in data:
    results.append(x**2)
print(len(results))
```

```python
def validate(d):
    if not isinstance(d, list):
        raise TypeError("Data must be a list")
    if not d:
        raise ValueError("Data list cannot be empty")
    if not all(isinstance(x, int) for x in d):
        raise TypeError("All elements must be integers")

try:
    d = [i for i in range(1, 1000000)]
    validate(d)
    res = [x**2 for x in d]
    print(len(res))
except (TypeError, ValueError) as e:
    print(f"Error: {e}")
```

OUTPUT:

```
PS C:\Users\chvis\OneDrive\Desktop\ai> & C:/Python314/python.exe "c:/Users/chvis/OneDrive/Desktop/ai/all g
Y/demo.py"
Y/demo.py"
999999
```

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

def grade(score):

if score >= 90:

return "A"

else:

if score >= 80:

return "B"

else:

if score >= 70:

return "C"

else:

if score >= 60:

return "D"

else:

return "F"

Expected Output:

• Cleaner logic using elif or dictionary mapping.

CODE:

```
def grade(s):
    if s >= 90:
        return "A"
    elif s >= 80:
        return "B"
    elif s >= 70:
        return "C"
    elif s >= 60:
        return "D"
    else:
        return "F"
```

```python
def g(s):
    """Return grade letter based on score using dictionary mapping."""
    m = {90: "A", 80: "B", 70: "C", 60: "D"}

    # Validate score input
    if not isinstance(s, (int, float)):
        raise TypeError("Score must be a number")
    if s < 0 or s > 100:
        raise ValueError("Score must be between 0 and 100")

    # Sort thresholds and find matching grade
    for t in sorted(m.keys(), reverse=True):
        if s >= t:
            return m[t]
    return "F"


def p(d):
    """Process multiple students and return their grades."""
    if not isinstance(d, dict):
        raise TypeError("Input must be a dictionary")

    r = {}
    for n, v in d.items():
        try:
            r[n] = g(v)
        except (TypeError, ValueError) as e:
            print(f"Error: {n} - {e}")
            r[n] = None
    return r
```

```python
def o(x):
    """Display grades in formatted table."""
    print("\n" + "="*40)
    print(f"{'Name':<20} {'Grade':<10}")
    print("="*40)

    for n, z in x.items():
        q = z if z else "Invalid"
        print(f"{n:<20} {q:<10}")

    print("="*40 + "\n")


# Main execution
try:
    d = {
        "Alice": 95,
        "Bob": 87,
        "Charlie": 72,
        "Diana": 65,
        "Eve": 58,
        "Frank": 91
    }

    h = p(d)
    o(h)

except Exception as e:
    print(f"Error: {e}")
```

OUTPUT:

```
========================================
Name                 Grade
========================================
Alice                A
Bob                  B
Charlie              C
Diana                D
Eve                  F
Frank                A
========================================
```